

COMPTE RENDU

SDN & Network Softwarization — TP3 — Topologies réseau et
datacenter avec Mininet, sans contrôleur
4e année Cybersécurité — École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2025/2026

Enseignante : Mme BOUHORMA

Date : 7 octobre 2026

Table des matières

Cadre du TP	2
1 Préparation de l'environnement	3
1.1 Test initial	4
2 Partie 1 — Topologies avec les commandes Mininet	5
2.1 Topologie Single — une étoile	5
2.2 Topologie Linear — une chaîne de switches	6
2.3 Topologie Tree — arbre à trois branches	8
2.4 Topologie Tree plus profonde	10
2.5 Comparer les topologies	12
3 Partie 2 — Topologies complexes créées avec Python	12
3.1 Structure minimale d'un programme Mininet	12
3.2 Architecture hiérarchique Core / Distribution / Access	13
4 Architecture Spine–Leaf	16
4.1 Vérification avec <code>sh ovs-vsctl show</code>	19
5 Architecture Fat-Tree $k = 4$	20
5.1 Vérification avec <code>sh ovs-vsctl show</code>	23
6 Tree vs Spine–Leaf vs Fat-Tree	24
7 Synthèse	25
8 Grille de validation rapide	26
A Sortie complète de <code>sh ovs-vsctl show</code> (Fat-Tree $k = 4$)	26

Cadre du TP

Plateforme	Outils	Mode
Ubuntu 24.04 LTS (portable Taha-inspiron-16)	Mininet 2.3.0, Open vSwitch 3.3.9, Python 3.12.3	Individuel
Approche	Switch / contrôleur	Livrable
Topologies construites en ligne de commande puis en Python	OVSBridge, aucun contrôleur SDN	Compte rendu avec captures

TABLE 1 – Cadre général du TP

Question fil rouge : *comment décrire, construire et tester des topologies réseau de plus en plus complexes (de l'étoile simple à l'architecture Fat-Tree d'un datacenter) avec Mininet, sans aucun contrôleur SDN ?*

Objectifs :

- lire et interpréter la forme d'une topologie Mininet (`nodes`, `net`, `links`, `dump`) ;
- créer plusieurs topologies avec la commande `mn`, sans écrire de Python ;
- créer des topologies structurées avec l'API Python de Mininet (`addHost`, `addSwitch`, `addLink`) ;
- comprendre les architectures hiérarchique, Spine-Leaf et Fat-Tree, modèles de datacenter ;
- tester la connectivité avec `pingall` et des `ping` ciblés.

Règle commune à tout le TP :

```
--switch ovsbr --controller none
```

OVSBridge fait fonctionner Open vSwitch comme un bridge Ethernet autonome : aucun contrôleur SDN n'est utilisé. Les topologies redondantes de la partie Python (Spine-Leaf, Fat-Tree) activent STP afin d'éviter les boucles de niveau 2.

Règle de travail :

- avant chaque nouvelle expérience : quitter Mininet avec `exit`, puis exécuter `sudo mn -c` pour nettoyer les anciens bridges et interfaces ;
- ne pas se limiter à « ping fonctionne » : relever ce que montrent `nodes`, `net` et `dump` et en tirer la structure de la topologie.

Note sur l'environnement. Le sujet est rédigé pour une VM Ubuntu. Toutes les manipulations de ce compte rendu ont été faites directement sur mon portable Ubuntu 24.04 (Taha-inspiron-16), dans le dossier `~/Bureau/UIR/4A/S7/SDN/TP3`. Les commandes sont identiques à celles du sujet.

1 Préparation de l'environnement

Listing 1 – Installation (sujet, partie 0) et vérifications

```
1 sudo apt update
2 sudo apt install -y mininet openvswitch-switch python3 python3-
  pip iperf curl traceroute net-tools iproute2
3 sudo systemctl enable --now openvswitch-switch
4
5 mn --version
6 python3 --version
7 ovs-vsctl --version
8 sudo systemctl status openvswitch-switch --no-pager
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ mn --version
2.3.0
```

FIGURE 1 – Sortie de mn --version

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ python3 --version
Python 3.12.3
```

FIGURE 2 – Sortie de python3 --version

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ ovs-vsctl --version
ovs-vsctl (Open vSwitch) 3.3.9
DB Schema 8.5.1
```

FIGURE 3 – Sortie de ovs-vsctl --version

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ sudo systemctl status openvswitch-switch --no-pager
● openvswitch-switch.service - Open vSwitch
   Loaded: loaded (/usr/lib/systemd/system/openvswitch-switch.service; enabled; preset: enabled)
   Active: active (exited) since Tue 2026-10-06 14:43:15 +00; 2min 5s ago
     Main PID: 34457 (code=exited, status=0/SUCCESS)
        CPU: 1ms

oct. 06 14:43:15 Taha-inspiron-16 systemd[1]: Starting openvswitch-switch.service - Open vSwitch...
oct. 06 14:43:15 Taha-inspiron-16 systemd[1]: Finished openvswitch-switch.service - Open vSwitch.
```

FIGURE 4 – Sortie de sudo systemctl status openvswitch-switch --no-pager

Outil	Version / état relevé
mn (Mininet)	2.3.0
python3	Python 3.12.3
ovs-vsctl (Open vSwitch)	3.3.9, schéma de base de données 8.5.1
openvswitch-switch	enabled, active (exited)

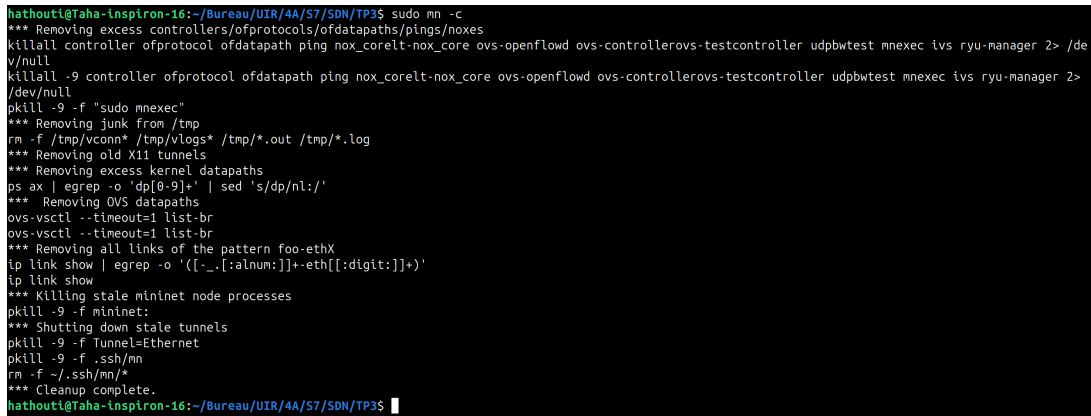
TABLE 2 – Versions des outils et état du service Open vSwitch

Les trois outils sont installés. Pour le service `openvswitch-switch`, l'état `active (exited)` est l'état normal : l'unité `systemd` se contente de lancer les démons `Open vSwitch` puis se termine. Elle est bien `enabled`, donc démarrée automatiquement au boot.

1.1 Test initial

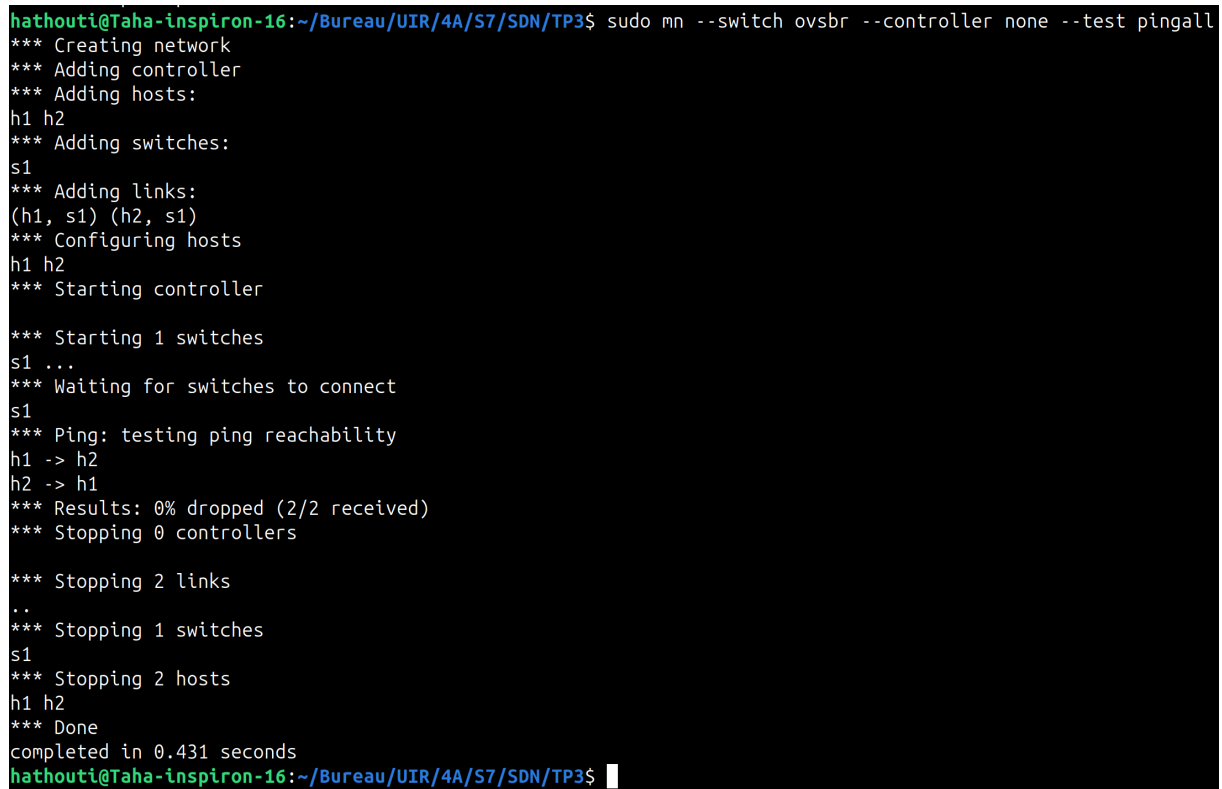
Listing 2 – Nettoyage puis test initial sans contrôleur

```
1 sudo mn -c
2 sudo mn --switch ovsbr --controller none --test pingall
```



```
hathouti@Taha-Inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ sudo mn -c
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | grep -o 'dp[0-9]*' | sed 's/dp/nl:/'
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | grep -o '([[:alnum:]]+-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn/*
rm -f ~/.ssh/mn/*
*** Cleanup complete.
hathouti@Taha-Inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$
```

FIGURE 5 – Nettoyage initial avec `sudo mn -c`



```
hathouti@Taha-Inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ sudo mn --switch ovsbr --controller none --test pingall
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
*** Stopping 0 controllers

*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 0.431 seconds
hathouti@Taha-Inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$
```

FIGURE 6 – Test initial : `sudo mn --switch ovsbr --controller none --test pingall`

Le test crée une topologie minimale (deux hôtes `h1`, `h2` et un switch `s1`), lance `pingall` puis arrête tout : `Results: 0% dropped (2/2 received)`, en 0,431 s. Mininet affiche toujours « Starting controller », mais aucun contrôleur n'est réellement lancé : l'arrêt indique `Stopping 0 controllers`, ce qui confirme l'effet de `--controller none`.

2 Partie 1 — Topologies avec les commandes Mininet

Dans cette partie, aucun script Python : Mininet construit la topologie demandée avec `--topo`. Chaque expérience commence par `sudo mn -c` et se lance avec `OVSBridge` sans contrôleur.

2.1 Topologie Single — une étoile

Listing 3 – Topologie Single

```
1 sudo mn -c
2 sudo mn --switch ovsbr --controller none --topo single,4
```

```
mininet> nodes
available nodes are:
h1 h2 h3 h4 s1
```

FIGURE 7 – Sortie de nodes

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0
```

FIGURE 8 – Sortie de net

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
```

FIGURE 9 – Sortie de pingall

```
mininet> h1 ping -c 3 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=0.357 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=0.084 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=0.089 ms

--- 10.0.0.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2053ms
rtt min/avg/max/mdev = 0.084/0.176/0.357/0.127 ms
```

FIGURE 10 – Sortie de `h1 ping -c 3 h4`

Observation. `nodes` liste cinq nœuds : les hôtes `h1` à `h4` et un seul switch `s1`. `net` montre que chaque hôte est branché sur un port différent de ce switch (`h1-eth0:s1-eth1`, ..., `h4-eth0:s1-eth4`) : tous les hôtes dépendent du même point central. `pingall` teste les $4 \times 3 = 12$ paires : 0% dropped (12/12 received).

Le ping `h1` → `h4` réussit (3/3, RTT moyen 0,176 ms, `t1=64`). Le premier paquet est plus lent (0,357 ms contre 0,084 et 0,089 ms) : il inclut la résolution ARP et l'apprentissage des adresses MAC par le switch ; les suivants profitent des caches. Le `t1=64` inchangé indique qu'aucun routeur n'est traversé.

```
mininet> exit
*** Stopping 0 controllers

*** Stopping 4 links
....
*** Stopping 1 switches
s1
*** Stopping 4 hosts
h1 h2 h3 h4
*** Done
completed in 65689.936 seconds
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ sudo mn -c
```

FIGURE 11 – Arrêt du réseau (`exit`) : 4 liens, 1 switch, 4 hôtes, puis `sudo mn -c`

2.2 Topologie Linear — une chaîne de switches

Listing 4 – Topologie Linear

```
1 sudo mn --switch ovsbr --controller none --topo linear,5
2 mininet> net
3 mininet> h1 ping -c 3 h5
4 mininet> links
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ sudo mn --switch ovsbr --controller none --topo linear,5
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5
*** Adding switches:
s1 s2 s3 s4 s5
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (h5, s5) (s2, s1) (s3, s2) (s4, s3) (s5, s4)
*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller

*** Starting 5 switches
s1 s2 s3 s4 s5 ...
*** Starting CLI:
mininet>
```

FIGURE 12 – Création de la topologie `linear,5` : liste des liens ajoutés par Mininet

```

mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s2-eth1
h3 h3-eth0:s3-eth1
h4 h4-eth0:s4-eth1
h5 h5-eth0:s5-eth1
s1 lo: s1-eth1:h1-eth0 s1-eth2:s2-eth2
s2 lo: s2-eth1:h2-eth0 s2-eth2:s1-eth2 s2-eth3:s3-eth2
s3 lo: s3-eth1:h3-eth0 s3-eth2:s2-eth3 s3-eth3:s4-eth2
s4 lo: s4-eth1:h4-eth0 s4-eth2:s3-eth3 s4-eth3:s5-eth2
s5 lo: s5-eth1:h5-eth0 s5-eth2:s4-eth3
mininet>

```

FIGURE 13 – Sortie de net

```

mininet> h1 ping -c 3 h5
PING 10.0.0.5 (10.0.0.5) 56(84) bytes of data.
64 bytes from 10.0.0.5: icmp_seq=1 ttl=64 time=1.85 ms
64 bytes from 10.0.0.5: icmp_seq=2 ttl=64 time=0.130 ms
64 bytes from 10.0.0.5: icmp_seq=3 ttl=64 time=0.155 ms

--- 10.0.0.5 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2047ms
rtt min/avg/max/mdev = 0.130/0.712/1.853/0.806 ms
mininet>

```

FIGURE 14 – Sortie de h1 ping -c 3 h5

```

mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s2-eth1 (OK OK)
h3-eth0<->s3-eth1 (OK OK)
h4-eth0<->s4-eth1 (OK OK)
h5-eth0<->s5-eth1 (OK OK)
s2-eth2<->s1-eth2 (OK OK)
s3-eth2<->s2-eth3 (OK OK)
s4-eth2<->s3-eth3 (OK OK)
s5-eth2<->s4-eth3 (OK OK)
mininet>

```

FIGURE 15 – Sortie de links

Observation. Mininet crée cinq hôtes et cinq switches : chaque switch s_i porte l'hôte h_i (liens $(h_1, s_1) \dots (h_5, s_5)$), et les switches sont reliés en chaîne par (s_2, s_1) (s_3, s_2) (s_4, s_3) (s_5, s_4) . links liste ces 9 liens, tous à l'état (OK OK).

Question : quels switches sont traversés entre h_1 et h_5 ?

À partir de net : h_1 est sur s_1 -eth1 ; s_1 -eth2 est relié à s_2 -eth2 ; s_2 -eth3 à s_3 -eth2 ; s_3 -eth3 à s_4 -eth2 ; s_4 -eth3 à s_5 -eth2 ; h_5 est sur s_5 -eth1. Le chemin est donc

$$h_1 \rightarrow s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_4 \rightarrow s_5 \rightarrow h_5$$

soit **cinq switches** traversés (les deux extrémités sont aux deux bouts de la chaîne). Le ping réussit (3/3, RTT moyen 0,712 ms) ; le premier paquet (1,85 ms) est nettement plus

lent que les suivants (0,130 et 0,155 ms) pour la même raison que précédemment, avec cette fois l'apprentissage MAC sur cinq switches.

```
mininet> exit
*** Stopping 0 controllers

*** Stopping 9 links
.....
*** Stopping 5 switches
s1 s2 s3 s4 s5
*** Stopping 5 hosts
h1 h2 h3 h4 h5
*** Done
completed in 254.271 seconds
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ sudo mn -c
```

FIGURE 16 – Arrêt du réseau : 9 liens, 5 switches, 5 hôtes

2.3 Topologie Tree — arbre à trois branches

Listing 5 – Topologie Tree, profondeur 2, fanout 3

```
1 sudo mn --switch ovsbr --controller none --topo tree,depth=2,
   fanout=3
2 mininet> nodes
3 mininet> net
4 mininet> pingall
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ sudo mn --switch ovsbr --controller none --topo tree,depth=2,fanout=3
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5 h6 h7 h8 h9
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(s1, s2) (s1, s3) (s1, s4) (s2, h1) (s2, h2) (s2, h3) (s3, h4) (s3, h5) (s3, h6) (s4, h7) (s4, h8) (s4, h9)
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8 h9
*** Starting controller

*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet>
```

FIGURE 17 – Création de la topologie tree,depth=2,fanout=3

```
mininet> nodes
available nodes are:
h1 h2 h3 h4 h5 h6 h7 h8 h9 s1 s2 s3 s4
mininet>
```

FIGURE 18 – Sortie de nodes

```
mininet> net
h1 h1-eth0:s2-eth1
h2 h2-eth0:s2-eth2
h3 h3-eth0:s2-eth3
h4 h4-eth0:s3-eth1
h5 h5-eth0:s3-eth2
h6 h6-eth0:s3-eth3
h7 h7-eth0:s4-eth1
h8 h8-eth0:s4-eth2
h9 h9-eth0:s4-eth3
s1 lo: s1-eth1:s2-eth4 s1-eth2:s3-eth4 s1-eth3:s4-eth4
s2 lo: s2-eth1:h1-eth0 s2-eth2:h2-eth0 s2-eth3:h3-eth0 s2-eth4:s1-eth1
s3 lo: s3-eth1:h4-eth0 s3-eth2:h5-eth0 s3-eth3:h6-eth0 s3-eth4:s1-eth2
s4 lo: s4-eth1:h7-eth0 s4-eth2:h8-eth0 s4-eth3:h9-eth0 s4-eth4:s1-eth3
mininet>
```

FIGURE 19 – Sortie de net

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8 h9
h2 -> h1 h3 h4 h5 h6 h7 h8 h9
h3 -> h1 h2 h4 h5 h6 h7 h8 h9
h4 -> h1 h2 h3 h5 h6 h7 h8 h9
h5 -> h1 h2 h3 h4 h6 h7 h8 h9
h6 -> h1 h2 h3 h4 h5 h7 h8 h9
h7 -> h1 h2 h3 h4 h5 h6 h8 h9
h8 -> h1 h2 h3 h4 h5 h6 h7 h9
h9 -> h1 h2 h3 h4 h5 h6 h7 h8
*** Results: 0% dropped (72/72 received)
mininet>
```

FIGURE 20 – Sortie de pingall

Observation. `depth` est la profondeur de l'arbre et `fanout` le nombre de branches par niveau. On obtient 4 switches et 9 hôtes :

- la racine `s1` possède trois ports vers le niveau inférieur (`s1-eth1:s2-eth4`, `s1-eth2:s3-eth4`, `s1-eth3:s4-eth4`);
- chaque switch du niveau inférieur porte trois hôtes : `s2` (`h1`, `h2`, `h3`), `s3` (`h4`, `h5`, `h6`) et `s4` (`h7`, `h8`, `h9`).

`pingall` teste $9 \times 8 = 72$ paires : 0% dropped (72/72 received). Entre deux hôtes de branches différentes (par exemple `h1` et `h9`), le trafic passe par `s2`, la racine `s1` puis `s4`.

```
*** Results: 0% dropped (72/72 received)
mininet> exit
*** Stopping 0 controllers

*** Stopping 12 links
.....
*** Stopping 4 switches
s1 s2 s3 s4
*** Stopping 9 hosts
h1 h2 h3 h4 h5 h6 h7 h8 h9
*** Done
completed in 189.001 seconds
hathouti@Taha-inspiron-16:~/Bureau/UITR/4A/S7/SDN/TP3$ sudo mn -c
```

FIGURE 21 – Arrêt du réseau : 12 liens, 4 switches, 9 hôtes

2.4 Topologie Tree plus profonde

Listing 6 – Topologie Tree, profondeur 3, fanout 2

```
1 sudo mn --switch ovsbr --controller none --topo tree,depth=3,  
    fanout=2  
2 mininet> nodes  
3 mininet> net  
4 mininet> h1 ping -c 3 h8  
5 mininet> dump
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ sudo mn --switch ovsbr --controller none --topo tree,depth=3,fanout=2  
*** Creating network  
*** Adding controller  
*** Adding hosts:  
h1 h2 h3 h4 h5 h6 h7 h8  
*** Adding switches:  
s1 s2 s3 s4 s5 s6 s7  
*** Adding links:  
(s1, s2) (s1, s5) (s2, s3) (s2, s4) (s3, h1) (s3, h2) (s4, h3) (s4, h4) (s5, s6) (s5, s7) (s6, h5) (s6, h6) (s7, h7) (s7, h8)  
*** Configuring hosts  
h1 h2 h3 h4 h5 h6 h7 h8  
*** Starting controller  
  
*** Starting 7 switches  
s1 s2 s3 s4 s5 s6 s7 ...  
*** Starting CLI:  
mininet>
```

FIGURE 22 – Création de la topologie tree,depth=3,fanout=2

```
mininet> nodes  
available nodes are:  
h1 h2 h3 h4 h5 h6 h7 h8 s1 s2 s3 s4 s5 s6 s7  
mininet>
```

FIGURE 23 – Sortie de nodes

```
mininet> net  
h1 h1-eth0:s3-eth1  
h2 h2-eth0:s3-eth2  
h3 h3-eth0:s4-eth1  
h4 h4-eth0:s4-eth2  
h5 h5-eth0:s6-eth1  
h6 h6-eth0:s6-eth2  
h7 h7-eth0:s7-eth1  
h8 h8-eth0:s7-eth2  
s1 lo: s1-eth1:s2-eth3 s1-eth2:s5-eth3  
s2 lo: s2-eth1:s3-eth3 s2-eth2:s4-eth3 s2-eth3:s1-eth1  
s3 lo: s3-eth1:h1-eth0 s3-eth2:h2-eth0 s3-eth3:s2-eth1  
s4 lo: s4-eth1:h3-eth0 s4-eth2:h4-eth0 s4-eth3:s2-eth2  
s5 lo: s5-eth1:s6-eth3 s5-eth2:s7-eth3 s5-eth3:s1-eth2  
s6 lo: s6-eth1:h5-eth0 s6-eth2:h6-eth0 s6-eth3:s5-eth1  
s7 lo: s7-eth1:h7-eth0 s7-eth2:h8-eth0 s7-eth3:s5-eth2  
mininet>
```

FIGURE 24 – Sortie de net

```

mininet> h1 ping -c 3 h8
PING 10.0.0.8 (10.0.0.8) 56(84) bytes of data.
64 bytes from 10.0.0.8: icmp_seq=1 ttl=64 time=2.33 ms
64 bytes from 10.0.0.8: icmp_seq=2 ttl=64 time=0.122 ms
64 bytes from 10.0.0.8: icmp_seq=3 ttl=64 time=0.087 ms

--- 10.0.0.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2017ms
rtt min/avg/max/mdev = 0.087/0.845/2.328/1.048 ms
mininet>

```

FIGURE 25 – Sortie de `h1 ping -c 3 h8`

```

mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=76749>
<Host h2: h2-eth0:10.0.0.2 pid=76751>
<Host h3: h3-eth0:10.0.0.3 pid=76753>
<Host h4: h4-eth0:10.0.0.4 pid=76755>
<Host h5: h5-eth0:10.0.0.5 pid=76757>
<Host h6: h6-eth0:10.0.0.6 pid=76759>
<Host h7: h7-eth0:10.0.0.7 pid=76761>
<Host h8: h8-eth0:10.0.0.8 pid=76763>
<OVSBridge s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None pid=76768>
<OVSBridge s2: lo:127.0.0.1,s2-eth1:None,s2-eth2:None,s2-eth3:None pid=76771>
<OVSBridge s3: lo:127.0.0.1,s3-eth1:None,s3-eth2:None,s3-eth3:None pid=76774>
<OVSBridge s4: lo:127.0.0.1,s4-eth1:None,s4-eth2:None,s4-eth3:None pid=76777>
<OVSBridge s5: lo:127.0.0.1,s5-eth1:None,s5-eth2:None,s5-eth3:None pid=76780>
<OVSBridge s6: lo:127.0.0.1,s6-eth1:None,s6-eth2:None,s6-eth3:None pid=76783>
<OVSBridge s7: lo:127.0.0.1,s7-eth1:None,s7-eth2:None,s7-eth3:None pid=76786>
mininet>

```

FIGURE 26 – Sortie de `dump`

Observation. Il s'agit d'un arbre binaire de profondeur 3 : 7 switches (`s1` à `s7`) et 8 hôtes (`h1` à `h8`). D'après `net`, la racine `s1` a deux fils `s2` et `s5`; `s2` a pour fils `s3` (hôtes `h1`, `h2`) et `s4` (hôtes `h3`, `h4`); `s5` a pour fils `s6` (`h5`, `h6`) et `s7` (`h7`, `h8`).

Un échange `h1` → `h8` remonte vers la racine puis redescend dans l'autre branche :

$$h1 \rightarrow s3 \rightarrow s2 \rightarrow s1 \rightarrow s5 \rightarrow s7 \rightarrow h8$$

soit cinq switches. Le ping réussit (3/3, RTT moyen 0,845 ms; premier paquet 2,33 ms).

`dump` donne les propriétés des nœuds : les hôtes ont les adresses 10.0.0.1 à 10.0.0.8 (une interface `hi-eth0` chacun) et les sept switches sont de type `OVSBridge`. Leurs ports (`si-ethn`) affichent `None` comme adresse : un switch de niveau 2 n'a pas d'adresse IP sur ses ports. La racine `s1` a 2 ports, les six autres switches en ont 3 (un vers le parent, deux vers les enfants ou les hôtes).

```

mininet> exit
*** Stopping 0 controllers

*** Stopping 14 links
.....
*** Stopping 7 switches
s1 s2 s3 s4 s5 s6 s7
*** Stopping 8 hosts
h1 h2 h3 h4 h5 h6 h7 h8
*** Done
completed in 159.118 seconds
hathouti@Taba-inspiron-16:~/Bureau/UTR/4A/S7/SDN/TP3$ sudo mn -c

```

FIGURE 27 – Arrêt du réseau : 14 liens, 7 switches, 8 hôtes

2.5 Comparer les topologies

Topologie	Switches	Hôtes	Liens	Switches traversés	Redondance
<code>single,4</code>	1	4	4	1	aucune
<code>linear,5</code>	5	5	9	5 (h1 → h5)	aucune
<code>tree,2,3</code>	4	9	12	3 (h1 → h9)	aucune
<code>tree,3,2</code>	7	8	14	5 (h1 → h8)	aucune

TABLE 3 – Comparaison des topologies de la partie 1 (nombres relevés à l’arrêt du réseau et avec `net`)

Pour chaque topologie, le nombre de liens est égal au nombre de nœuds moins un (par exemple $4 + 1 = 5$ nœuds et 4 liens pour `single,4`; $7 + 8 = 15$ nœuds et 14 liens pour `tree,3,2`) : ce sont des graphes sans cycle, donc sans chemin de secours. C’est aussi pourquoi `OVSBridge` fonctionne sans STP dans toute cette partie.

Topologie	Organisation	Intérêt pédagogique	Limite principale
Single	Étoile	Comprendre la relation hôte ↔ switch	Un seul switch central
Linear	Chaîne	Observer un chemin multi-switch	Chemins longs
Tree	Hierarchie	Comprendre niveaux et agrégation	Pas de redondance

TABLE 4 – Single, Linear et Tree : intérêt et limites

3 Partie 2 — Topologies complexes créées avec Python

Python ne sert ici qu’à décrire l’architecture : ajouter des hôtes, des switches et des liens. Il n’y a toujours aucun contrôleur SDN.

3.1 Structure minimale d’un programme Mininet

Listing 7 – Squelette d’un script Mininet sans contrôleur

```

1 from mininet.net import Mininet
2 from mininet.node import OVSBridge
3 from mininet.cli import CLI
4
5 net = Mininet(controller=None, switch=OVSBridge)
6 # net.addHost(...)
7 # net.addSwitch(...)
8 # net.addLink(...)

```

```

9 net.start()
10 CLI(net)
11 net.stop()

```

- `controller=None` : aucun contrôleur n'est créé;
- `switch=OVSBridge` : les switches sont des bridges Ethernet Open vSwitch autonomes;
- `addHost`, `addSwitch`, `addLink` décrivent la topologie; `net.start()` la construit, `CLI(net)` ouvre la console Mininet et `net.stop()` détruit tout à la sortie.

Chaque script se lance avec :

Listing 8 – Exécution d'un script

```

1 sudo mn -c
2 sudo python3 nom_du_script.py

```

3.2 Architecture hiérarchique Core / Distribution / Access

Cette topologie reproduit une organisation classique à trois niveaux : un cœur (c1), deux switches de distribution (d1, d2), quatre switches d'accès (a1 à a4) et huit hôtes. Elle ne contient pas de boucle : STP n'est donc pas nécessaire.

Listing 9 – Script `hierarchie.py`

```

1 from mininet.net import Mininet
2 from mininet.node import OVSBridge
3 from mininet.cli import CLI
4 from mininet.log import setLogLevel, info
5
6 def build_network():
7     net = Mininet(controller=None, switch=OVSBridge)
8
9     core = net.addSwitch('c1')
10    d1 = net.addSwitch('d1')
11    d2 = net.addSwitch('d2')
12    net.addLink(core, d1)
13    net.addLink(core, d2)
14
15    access = []
16    for i, dist in enumerate((d1, d1, d2, d2), start=1):
17        sw = net.addSwitch(f'a{i}')
18        access.append(sw)
19        net.addLink(dist, sw)
20
21    host_id = 1
22    for sw in access:
23        for _ in range(2):
24            h = net.addHost(f'h{host_id}')
25            net.addLink(h, sw)
26            host_id += 1
27

```

```

28     info('***_Demarrage_du_reseau\n')
29     net.start()
30     net.pingAll()
31     CLI(net)
32     net.stop()
33
34 if __name__ == '__main__':
35     setLogLevel('info')
36     build_network()

```

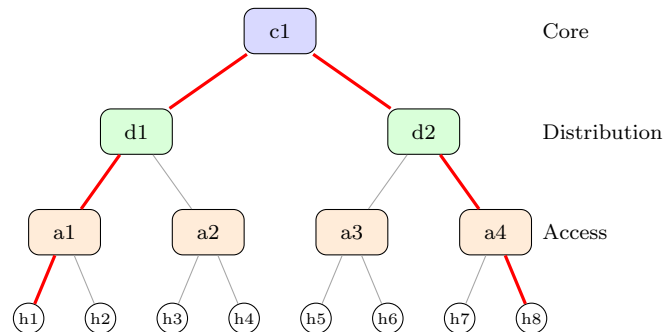


FIGURE 28 – Architecture hiérarchique; en rouge, le chemin $h1 \rightarrow h8$

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ sudo python3 hierarchie.py
*** Demarrage du reseau
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8
*** Starting controller

*** Starting 7 switches
c1 d1 d2 a1 a2 a3 a4 ...
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8
h2 -> h1 h3 h4 h5 h6 h7 h8
h3 -> h1 h2 h4 h5 h6 h7 h8
h4 -> h1 h2 h3 h5 h6 h7 h8
h5 -> h1 h2 h3 h4 h6 h7 h8
h6 -> h1 h2 h3 h4 h5 h7 h8
h7 -> h1 h2 h3 h4 h5 h6 h8
h8 -> h1 h2 h3 h4 h5 h6 h7
*** Results: 0% dropped (56/56 received)
*** Starting CLI:
mininet>

```

FIGURE 29 – Exécution de `sudo python3 hierarchie.py`

```

mininet> nodes
available nodes are:
a1 a2 a3 a4 c1 d1 d2 h1 h2 h3 h4 h5 h6 h7 h8
mininet>

```

FIGURE 30 – Sortie de `nodes`

```

mininet> net
h1 h1-eth0:a1-eth2
h2 h2-eth0:a1-eth3
h3 h3-eth0:a2-eth2
h4 h4-eth0:a2-eth3
h5 h5-eth0:a3-eth2
h6 h6-eth0:a3-eth3
h7 h7-eth0:a4-eth2
h8 h8-eth0:a4-eth3
c1 lo: c1-eth1:d1-eth1 c1-eth2:d2-eth1
d1 lo: d1-eth1:c1-eth1 d1-eth2:a1-eth1 d1-eth3:a2-eth1
d2 lo: d2-eth1:c1-eth2 d2-eth2:a3-eth1 d2-eth3:a4-eth1
a1 lo: a1-eth1:d1-eth2 a1-eth2:h1-eth0 a1-eth3:h2-eth0
a2 lo: a2-eth1:d1-eth3 a2-eth2:h3-eth0 a2-eth3:h4-eth0
a3 lo: a3-eth1:d2-eth2 a3-eth2:h5-eth0 a3-eth3:h6-eth0
a4 lo: a4-eth1:d2-eth3 a4-eth2:h7-eth0 a4-eth3:h8-eth0
mininet>

```

FIGURE 31 – Sortie de net

```

mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8
h2 -> h1 h3 h4 h5 h6 h7 h8
h3 -> h1 h2 h4 h5 h6 h7 h8
h4 -> h1 h2 h3 h5 h6 h7 h8
h5 -> h1 h2 h3 h4 h6 h7 h8
h6 -> h1 h2 h3 h4 h5 h7 h8
h7 -> h1 h2 h3 h4 h5 h6 h8
h8 -> h1 h2 h3 h4 h5 h6 h7
*** Results: 0% dropped (56/56 received)
mininet>

```

FIGURE 32 – Sortie de pingall

```

mininet> h1 ping -c 3 h8
PING 10.0.0.8 (10.0.0.8) 56(84) bytes of data.
64 bytes from 10.0.0.8: icmp_seq=1 ttl=64 time=2.24 ms
64 bytes from 10.0.0.8: icmp_seq=2 ttl=64 time=0.156 ms
64 bytes from 10.0.0.8: icmp_seq=3 ttl=64 time=0.206 ms

--- 10.0.0.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2059ms
rtt min/avg/max/mdev = 0.156/0.865/2.235/0.968 ms
mininet>

```

FIGURE 33 – Sortie de h1 ping -c 3 h8

Observation. Le script démarre 7 switches (c1 d1 d2 a1 a2 a3 a4) et lance lui-même un pingall avant d'ouvrir la CLI : 0% dropped (56/56 received), soit 8×7 paires. nodes confirme les 15 nœuds (7 switches, 8 hôtes). net donne la hiérarchie : c1 est relié à d1 et d2 ; d1 porte a1 et a2 ; d2 porte a3 et a4 ; chaque switch d'accès porte deux hôtes (a1 : h1, h2 ; ... ; a4 : h7, h8). Le ping h1 → h8 réussit (3/3, RTT moyen 0,865 ms).

Question : quel est le rôle de chaque niveau ? Quel est le chemin logique h1 → h8 ?

Niveau	Switches	Rôle
Core	c1	Interconnecter les switches de distribution : tout trafic entre deux branches y passe.
Distribution	d1, d2	Agréger les switches d'accès et relier chaque groupe au cœur.
Access	a1 à a4	Raccorder directement les hôtes (deux par switch).

TABLE 5 – Rôle des trois niveaux

D'après `net`, le chemin logique est

$$h1 \rightarrow a1 \rightarrow d1 \rightarrow c1 \rightarrow d2 \rightarrow a4 \rightarrow h8$$

(`h1` et `h8` étant sur deux branches différentes, le trafic monte jusqu'au cœur puis redescend, comme dans un arbre).

4 Architecture Spine–Leaf

Dans une architecture Spine–Leaf, chaque switch *leaf* est relié à tous les switches *spine*, et les serveurs sont connectés aux leaves. Entre deux racks, le trafic traverse au plus leaf, spine, leaf, et plusieurs chemins physiques existent. Comme cette topologie contient des boucles de niveau 2 et qu'aucun contrôleur n'est présent, le script active STP.

Listing 10 – Script `spine_leaf.py`

```

1 from mininet.net import Mininet
2 from mininet.node import OVSBridge
3 from mininet.cli import CLI
4 from mininet.log import setLogLevel, info
5 from time import sleep
6
7 class STPBridge(OVSBridge):
8     def __init__(self, name, **kwargs):
9         super().__init__(name, stp=True, **kwargs)
10
11 def build_network():
12     net = Mininet(controller=None, switch=STPBridge)
13
14     info('*** Creation des spines\n')
15     sp1 = net.addSwitch('sp1')
16     sp2 = net.addSwitch('sp2')
17
18     info('*** Creation des leaves et des hosts\n')
19     leaves = []
20     for i in range(1, 5):
21         leaf = net.addSwitch(f'l{i}')
22         leaves.append(leaf)
23         h1 = net.addHost(f'h{2*i-1}')
24         h2 = net.addHost(f'h{2*i}')
25         net.addLink(h1, leaf)
26         net.addLink(h2, leaf)

```

```

27     net.addLink(leaf, sp1)
28     net.addLink(leaf, sp2)
29
30     info('***_Demarrage_du_reseau\n')
31     net.start()
32     info('***_STP:_attendre_la_convergence_avant_pingall_(~35_s)
33         \n')
34     sleep(35)
35     net.pingAll()
36     CLI(net)
37     net.stop()
38
39 if __name__ == '__main__':
40     setLogLevel('info')
41     build_network()

```

La classe STPBridge hérite de OVSBridge et force `stp=True` à la création de chaque switch : Open vSwitch émet alors des BPDU, élit un switch racine et bloque les liens redondants pour ne conserver qu'un arbre actif. Les ports passent par les états *listening* puis *learning* avant de transmettre (environ 30 s avec les temporisations par défaut). C'est pourquoi le script attend 35 s (`sleep(35)`) avant de lancer `pingAll()` : un test immédiat donnerait des pertes.

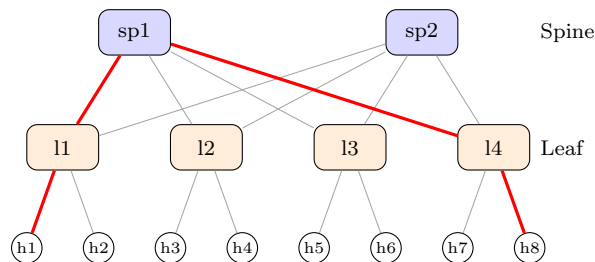


FIGURE 34 – Architecture Spine-Leaf; en rouge, un chemin possible $h1 \rightarrow h8$

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ sudo python3 spine_leaf.py
*** Creation des spines
*** Creation des leaves et des hosts
*** Demarrage du reseau
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8
*** Starting controller

*** Starting 6 switches
sp1 sp2 l1 l2 l3 l4 ...
*** STP : attendre la convergence avant pingall (~35 s)
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8
h2 -> h1 h3 h4 h5 h6 h7 h8
h3 -> h1 h2 h4 h5 h6 h7 h8
h4 -> h1 h2 h3 h5 h6 h7 h8
h5 -> h1 h2 h3 h4 h6 h7 h8
h6 -> h1 h2 h3 h4 h5 h7 h8
h7 -> h1 h2 h3 h4 h5 h6 h8
h8 -> h1 h2 h3 h4 h5 h6 h7
*** Results: 0% dropped (56/56 received)
*** Starting CLI:
mininet>

```

FIGURE 35 – Exécution de `sudo python3 spine_leaf.py`

```
mininet> nodes
available nodes are:
h1 h2 h3 h4 h5 h6 h7 h8 l1 l2 l3 l4 sp1 sp2
mininet>
```

FIGURE 36 – Sortie de nodes

```
mininet> net
h1 h1-eth0:l1-eth1
h2 h2-eth0:l1-eth2
h3 h3-eth0:l2-eth1
h4 h4-eth0:l2-eth2
h5 h5-eth0:l3-eth1
h6 h6-eth0:l3-eth2
h7 h7-eth0:l4-eth1
h8 h8-eth0:l4-eth2
sp1 lo: sp1-eth1:l1-eth3 sp1-eth2:l2-eth3 sp1-eth3:l3-eth3 sp1-eth4:l4-eth3
sp2 lo: sp2-eth1:l1-eth4 sp2-eth2:l2-eth4 sp2-eth3:l3-eth4 sp2-eth4:l4-eth4
l1 lo: l1-eth1:h1-eth0 l1-eth2:h2-eth0 l1-eth3:sp1-eth1 l1-eth4:sp2-eth1
l2 lo: l2-eth1:h3-eth0 l2-eth2:h4-eth0 l2-eth3:sp1-eth2 l2-eth4:sp2-eth2
l3 lo: l3-eth1:h5-eth0 l3-eth2:h6-eth0 l3-eth3:sp1-eth3 l3-eth4:sp2-eth3
l4 lo: l4-eth1:h7-eth0 l4-eth2:h8-eth0 l4-eth3:sp1-eth4 l4-eth4:sp2-eth4
mininet>
```

FIGURE 37 – Sortie de net

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8
h2 -> h1 h3 h4 h5 h6 h7 h8
h3 -> h1 h2 h4 h5 h6 h7 h8
h4 -> h1 h2 h3 h5 h6 h7 h8
h5 -> h1 h2 h3 h4 h6 h7 h8
h6 -> h1 h2 h3 h4 h5 h7 h8
h7 -> h1 h2 h3 h4 h5 h6 h8
h8 -> h1 h2 h3 h4 h5 h6 h7
*** Results: 0% dropped (56/56 received)
mininet>
```

FIGURE 38 – Sortie de pingall

```
mininet> h1 ping -c 3 h8
PING 10.0.0.8 (10.0.0.8) 56(84) bytes of data.
64 bytes from 10.0.0.8: icmp_seq=1 ttl=64 time=1.55 ms
64 bytes from 10.0.0.8: icmp_seq=2 ttl=64 time=0.089 ms
64 bytes from 10.0.0.8: icmp_seq=3 ttl=64 time=0.094 ms

--- 10.0.0.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2052ms
rtt min/avg/max/mdev = 0.089/0.578/1.553/0.688 ms
mininet>
```

FIGURE 39 – Sortie de h1 ping -c 3 h8

Observation. Le script affiche le message d'attente STP, puis le pingall automatique donne 0% dropped (56/56 received) : la convergence a bien eu lieu avant le test. nodes liste 14 nœuds : 8 hôtes, 4 leaves (l1 à l4) et 2 spines (sp1, sp2). D'après net, chaque leaf a quatre ports : deux vers ses hôtes (par exemple l1-eth1:h1-eth0 et

l1-eth2:h2-eth0) et deux liens montants, l'un vers sp1 (l1-eth3:sp1-eth1) et l'autre vers sp2 (l1-eth4:sp2-eth1). Chaque spine a quatre ports, un vers chaque leaf. Le ping h1 → h8 réussit (3/3, RTT moyen 0,578 ms).

4.1 Vérification avec `sh ovs-vsctl show`

<pre> Bridge l1 fail_mode: standalone Port l1-eth4 Interface l1-eth4 Port l1-eth1 Interface l1-eth1 Port l1 Interface l1 type: internal Port l1-eth3 Interface l1-eth3 Port l1-eth2 Interface l1-eth2 </pre>	<pre> Bridge l2 fail_mode: standalone Port l2-eth4 Interface l2-eth4 Port l2 Interface l2 type: internal Port l2-eth1 Interface l2-eth1 Port l2-eth3 Interface l2-eth3 Port l2-eth2 Interface l2-eth2 </pre>	<pre> Bridge l3 fail_mode: standalone Port l3 Interface l3 type: internal Port l3-eth1 Interface l3-eth1 Port l3-eth4 Interface l3-eth4 Port l3-eth3 Interface l3-eth3 Port l3-eth2 Interface l3-eth2 </pre>
l1	l2	l3
<pre> Bridge l4 fail_mode: standalone Port l4-eth1 Interface l4-eth1 Port l4-eth2 Interface l4-eth2 Port l4-eth4 Interface l4-eth4 Port l4-eth3 Interface l4-eth3 Port l4 Interface l4 type: internal </pre>	<pre> Bridge sp1 fail_mode: standalone Port sp1-eth4 Interface sp1-eth4 Port sp1 Interface sp1 type: internal Port sp1-eth3 Interface sp1-eth3 Port sp1-eth1 Interface sp1-eth1 Port sp1-eth2 Interface sp1-eth2 </pre>	<pre> Bridge sp2 fail_mode: standalone Port sp2-eth4 Interface sp2-eth4 Port sp2-eth3 Interface sp2-eth3 Port sp2-eth2 Interface sp2-eth2 Port sp2-eth1 Interface sp2-eth1 Port sp2 Interface sp2 type: internal </pre>
l4	sp1	sp2

FIGURE 40 – Extraits de `sh ovs-vsctl show` pour les six bridges de la topologie Spine-Leaf

Chaque bridge est en `fail_mode: standalone` : il se comporte comme un switch Ethernet classique, sans dépendre d'un contrôleur. Chacun possède un port interne portant son propre nom (`type: internal`) et quatre ports `eth1` à `eth4`, ce qui correspond bien à ce que montre `net`. Cette commande n'affiche pas l'état STP des ports ; pour le consulter, on pourrait utiliser `ovs-appctl stp/show`.

Questions : combien de spines ? de leaves ? de hôtes par leaf ? Pourquoi chaque leaf possède-t-il deux uplinks ?

- **2 spines** (sp1, sp2), **4 leaves** (l1 à l4) et **2 hôtes par leaf**, soit 8 hôtes au total ;
- chaque leaf est relié aux deux spines par deux uplinks afin de disposer de **deux chemins physiques** vers n'importe quel autre leaf : si un spine ou un lien tombe, l'autre assure la continuité, et le nombre de switches traversés reste constant (leaf, spine, leaf). Avec STP, un seul de ces deux chemins est utilisé à un instant donné, l'autre est bloqué en attente.

5 Architecture Fat-Tree $k = 4$

Le Fat-Tree est une topologie Clos hiérarchique construite pour multiplier les chemins entre les serveurs. Pour $k = 4$: 4 pods, 4 switches core, 8 switches d'agrégation, 8 switches edge et 16 hôtes. Chaque edge relie deux hôtes et les deux switches d'agrégation de son pod ; le premier agrégat de chaque pod est relié aux cores `c1` et `c2`, le second à `c3` et `c4`.

Niveau	Nombre	Rôle
Core	4	Interconnecter les pods
Aggregation	8	Relier edge et core
Edge	8	Connecter les serveurs
Hosts	16	Serveurs émulés

TABLE 6 – Organisation du Fat-Tree $k = 4$

Listing 11 – Script `fattree_k4.py`

```

1 from mininet.net import Mininet
2 from mininet.node import OVSBridge
3 from mininet.cli import CLI
4 from mininet.log import setLogLevel, info
5 from time import sleep
6
7 class STPBridge(OVSBridge):
8     def __init__(self, name, **kwargs):
9         super().__init__(name, stp=True, **kwargs)
10
11 def build_fattree_k4():
12     net = Mininet(controller=None, switch=STPBridge)
13
14     # k = 4 : 4 core, 8 aggregation, 8 edge, 16 hosts
15     core = [net.addSwitch(f'c{i}') for i in range(1, 5)]
16
17     host_id = 1
18     for pod in range(1, 5):
19         aggs = [net.addSwitch(f'a{pod}{j}') for j in range(1, 3)]
20         edges = [net.addSwitch(f'e{pod}{j}') for j in range(1, 3)]
21
22         # Chaque edge est relie aux 2 aggregation du pod
23         for edge in edges:
24             for agg in aggs:
25                 net.addLink(edge, agg)
26
27         # 2 hosts par edge
28         for _ in range(2):
29             host = net.addHost(f'h{host_id}')
30             net.addLink(host, edge)
31             host_id += 1
32

```

```

33      # Agg 1 -> groupe de core 1 ; Agg 2 -> groupe de core 2
34      net.addLink(aggs[0], core[0])
35      net.addLink(aggs[0], core[1])
36      net.addLink(aggs[1], core[2])
37      net.addLink(aggs[1], core[3])
38
39      info('***_Demarrage_Fat-Tree_k=4\n')
40      net.start()
41      info('***_STP:_attendre_la_convergence_avant_pingall_(~40_s)
42          \n')
43      sleep(40)
44      net.pingAll()
45      CLI(net)
46      net.stop()
47
48 if __name__ == '__main__':
49     setLogLevel('info')
50     build_fattree_k4()

```

Comme pour le Spine-Leaf, la classe `STPBridge` active STP sur les 20 switches et le script attend 40 s avant `pingAll()`.

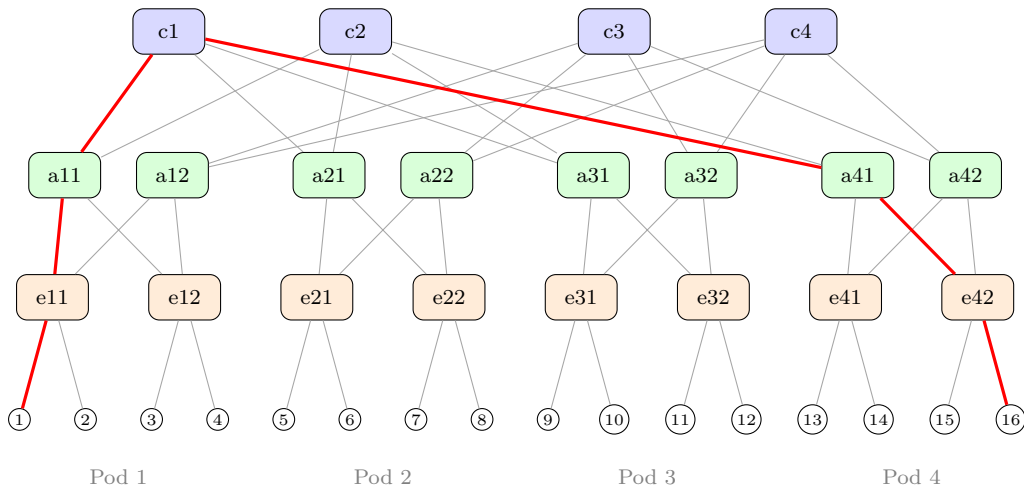


FIGURE 41 – Fat-Tree $k = 4$ (les hôtes sont numérotés de 1 à 16); en rouge, un chemin $h_1 \rightarrow h_{16}$

```

hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP3$ sudo python3 fattree_k4.py
*** Demarrage Fat-Tree k=4
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
*** Starting controller

*** Starting 20 switches
c1 c2 c3 c4 a11 a12 e11 e12 a21 a22 e21 e22 a31 a32 e31 e32 a41 a42 e41 e42 ...
*** STP : attendre la convergence avant pingall (~40 s)
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
h2 -> h1 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
h3 -> h1 h2 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
h4 -> h1 h2 h3 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
h5 -> h1 h2 h3 h4 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
h6 -> h1 h2 h3 h4 h5 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
h7 -> h1 h2 h3 h4 h5 h6 h8 h9 h10 h11 h12 h13 h14 h15 h16
h8 -> h1 h2 h3 h4 h5 h6 h7 h9 h10 h11 h12 h13 h14 h15 h16
h9 -> h1 h2 h3 h4 h5 h6 h7 h8 h10 h11 h12 h13 h14 h15 h16
h10 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h11 h12 h13 h14 h15 h16
h11 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h12 h13 h14 h15 h16
h12 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h13 h14 h15 h16
h13 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h14 h15 h16
h14 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h15 h16
h15 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h16
h16 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15
*** Results: 0% dropped (240/240 received)
*** Starting CLI:
mininet>

```

FIGURE 42 – Exécution de `sudo python3 fattree_k4.py`

```

mininet> nodes
available nodes are:
a11 a12 a21 a22 a31 a32 a41 a42 c1 c2 c3 c4 e11 e12 e21 e22 e31 e32 e41 e42 h1 h10 h11 h12 h13 h14 h15 h16 h2 h3 h4 h5 h6 h7 h8 h9
mininet>

```

FIGURE 43 – Sortie de `nodes`

```

mininet> net
h1 h1-eth0:e11-eth3
h2 h2-eth0:e11-eth4
h3 h3-eth0:e12-eth3
h4 h4-eth0:e12-eth4
h5 h5-eth0:e21-eth3
h6 h6-eth0:e21-eth4
h7 h7-eth0:e22-eth3
h8 h8-eth0:e22-eth4
h9 h9-eth0:e31-eth3
h10 h10-eth0:e31-eth4
h11 h11-eth0:e32-eth3
h12 h12-eth0:e32-eth4
h13 h13-eth0:e41-eth3
h14 h14-eth0:e41-eth4
h15 h15-eth0:e42-eth3
h16 h16-eth0:e42-eth4
c1 lo: c1-eth1:a11-eth3 c1-eth2:a21-eth3 c1-eth3:a31-eth3 c1-eth4:a41-eth3
c2 lo: c2-eth1:a11-eth4 c2-eth2:a21-eth4 c2-eth3:a31-eth4 c2-eth4:a41-eth4
c3 lo: c3-eth1:a12-eth3 c3-eth2:a22-eth3 c3-eth3:a32-eth3 c3-eth4:a42-eth3
c4 lo: c4-eth1:a12-eth4 c4-eth2:a22-eth4 c4-eth3:a32-eth4 c4-eth4:a42-eth4
a11 lo: a11-eth1:e11-eth1 a11-eth2:e12-eth1 a11-eth3:c1-eth1 a11-eth4:c2-eth1
a12 lo: a12-eth1:e11-eth2 a12-eth2:e12-eth2 a12-eth3:c3-eth1 a12-eth4:c4-eth1
e11 lo: e11-eth1:a11-eth1 e11-eth2:a12-eth1 e11-eth3:h1-eth0 e11-eth4:h2-eth0
e12 lo: e12-eth1:a11-eth2 e12-eth2:a12-eth2 e12-eth3:h3-eth0 e12-eth4:h4-eth0
a21 lo: a21-eth1:e21-eth1 a21-eth2:e22-eth1 a21-eth3:c1-eth2 a21-eth4:c2-eth2
a22 lo: a22-eth1:e21-eth2 a22-eth2:e22-eth2 a22-eth3:c3-eth2 a22-eth4:c4-eth2
e21 lo: e21-eth1:a21-eth1 e21-eth2:a22-eth1 e21-eth3:h5-eth0 e21-eth4:h6-eth0
e22 lo: e22-eth1:a21-eth2 e22-eth2:a22-eth2 e22-eth3:h7-eth0 e22-eth4:h8-eth0
a31 lo: a31-eth1:e31-eth1 a31-eth2:e32-eth1 a31-eth3:c1-eth3 a31-eth4:c2-eth3
a32 lo: a32-eth1:e31-eth2 a32-eth2:e32-eth2 a32-eth3:c3-eth3 a32-eth4:c4-eth3
e31 lo: e31-eth1:a31-eth1 e31-eth2:a32-eth1 e31-eth3:h9-eth0 e31-eth4:h10-eth0
e32 lo: e32-eth1:a31-eth2 e32-eth2:a32-eth2 e32-eth3:h11-eth0 e32-eth4:h12-eth0
a41 lo: a41-eth1:e41-eth1 a41-eth2:e42-eth1 a41-eth3:c1-eth4 a41-eth4:c2-eth4
a42 lo: a42-eth1:e41-eth2 a42-eth2:e42-eth2 a42-eth3:c3-eth4 a42-eth4:c4-eth4
e41 lo: e41-eth1:a41-eth1 e41-eth2:a42-eth1 e41-eth3:h13-eth0 e41-eth4:h14-eth0
e42 lo: e42-eth1:a41-eth2 e42-eth2:a42-eth2 e42-eth3:h15-eth0 e42-eth4:h16-eth0
mininet>

```

FIGURE 44 – Sortie de `net`

```

mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
h2 -> h1 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
h3 -> h1 h2 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
h4 -> h1 h2 h3 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
h5 -> h1 h2 h3 h4 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
h6 -> h1 h2 h3 h4 h5 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
h7 -> h1 h2 h3 h4 h5 h6 h8 h9 h10 h11 h12 h13 h14 h15 h16
h8 -> h1 h2 h3 h4 h5 h6 h7 h9 h10 h11 h12 h13 h14 h15 h16
h9 -> h1 h2 h3 h4 h5 h6 h7 h8 h10 h11 h12 h13 h14 h15 h16
h10 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h11 h12 h13 h14 h15 h16
h11 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h12 h13 h14 h15 h16
h12 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h13 h14 h15 h16
h13 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h14 h15 h16
h14 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h15 h16
h15 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h16
h16 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15
*** Results: 0% dropped (240/240 received)
mininet>

```

FIGURE 45 – Sortie de pingall

```

mininet> h1 ping -c 3 h16
PING 10.0.0.16 (10.0.0.16) 56(84) bytes of data.
64 bytes from 10.0.0.16: icmp_seq=1 ttl=64 time=3.06 ms
64 bytes from 10.0.0.16: icmp_seq=2 ttl=64 time=0.120 ms
64 bytes from 10.0.0.16: icmp_seq=3 ttl=64 time=0.171 ms

--- 10.0.0.16 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2047ms
rtt min/avg/max/mdev = 0.120/1.118/3.064/1.375 ms
mininet>

```

FIGURE 46 – Sortie de h1 ping -c 3 h16

Observation. Le script démarre 20 switches et attend la convergence de STP; le pingall automatique puis celui de la CLI donnent 0% dropped (240/240 received), soit 16×15 paires. nodes affiche bien les 36 nœuds attendus : 4 cores (c1 à c4), 8 agrégations (a11 à a42), 8 edges (e11 à e42) et 16 hôtes. Le nom d'un switch encode son pod : a21 est l'agrégation 1 du pod 2, e42 l'edge 2 du pod 4. D'après net :

- chaque edge porte deux hôtes (e11 : h1, h2; e12 : h3, h4; ...; e42 : h15, h16) et est relié aux deux agrégations de son pod;
- a11 est relié à c1 (c1-eth1:a11-eth3) et c2 (c2-eth1:a11-eth4); a12 est relié à c3 et c4;
- chaque core possède quatre ports, un vers chaque pod (par exemple c1 : a11, a21, a31, a41).

Le ping h1 → h16 réussit (3/3, RTT moyen 1,118 ms; premier paquet 3,06 ms contre 0,120 et 0,171 ms). Le ttl=64 inchangé confirme qu'aucun routeur n'intervient : toute la communication reste au niveau 2.

5.1 Vérification avec sh ovs-vsctl show

La sortie complète de sh ovs-vsctl show, trop longue pour une capture, est reproduite en annexe A. Elle contient 20 bridges, tous en fail_mode: standalone, chacun avec un port interne et quatre ports eth1 à eth4.

Niveau	Bridges	Ports eth par bridge	Extrémités de liens
Core	c1 à c4	4 (un par pod)	16
Aggregation	a11 à a42	4 (2 edges, 2 cores)	32
Edge	e11 à e42	4 (2 agrégations, 2 hôtes)	32

TABLE 7 – Ports relevés sur les 20 bridges du Fat-Tree

Cohérence : $20 \times 4 = 80$ extrémités de liens côté switch, plus 16 côté hôtes, soit 96 extrémités, c'est-à-dire 48 liens (16 hôte-edge, 16 edge-agrégation et 16 agrégation-core).

Travail : identifier le pod de h1 et celui de h16, puis reconstituer un chemin possible.

h1 est branché sur e11 : il appartient au **pod 1**. h16 est branché sur e42 : il appartient au **pod 4**. Un chemin possible, en distinguant les niveaux, est :

h1 → e11 (Edge) → a11 (Aggregation) → c1 (Core) → a41 (Aggregation) → e42
(Edge) → h16

Ce chemin est valide d'après net : e11-eth1:a11-eth1, a11-eth3:c1-eth1, c1-eth4:a41-eth3 et a41-eth2:e42-eth1. Il en existe trois autres de même longueur, soit quatre chemins entre ces deux pods : a11 → c2 → a41, ainsi que, par la seconde agrégation de chaque pod, a12 → c3 → a42 et a12 → c4 → a42. Avec STP, un seul de ces chemins reste actif.

6 Tree vs Spine-Leaf vs Fat-Tree

Architecture	Structure	Switches / hôtes / liens	Switches traversés	Redondance	Usage
Tree / hiérarchique	Hiérarchie simple	7 / 8 / 14	5 (h1 → h8)	Faible (aucune)	Comprendre les niveaux
Spine-Leaf	Leaves reliés à tous les spines	6 / 8 / 16	3 (h1 → h8)	Élevée : 2 chemins entre leaves	Datacenter moderne, chemins courts
Fat-Tree $k = 4$	Pods + edge + aggregation + core	20 / 16 / 48	5 (h1 → h16)	Très élevée : 4 chemins entre pods	Comprendre Clos et multipath

TABLE 8 – Comparaison des trois architectures (valeurs tirées des sorties net)

Dans l'arbre, il n'existe qu'un chemin entre deux hôtes ; le Spine-Leaf raccourcit les chemins (trois switches au plus) et en offre deux ; le Fat-Tree multiplie les chemins (quatre entre pods différents) au prix de beaucoup plus de switches et de liens.

Limite sans contrôleur. STP ne conserve qu'un arbre actif : avec N switches, seuls $N - 1$ liens entre switches restent actifs. Pour le Spine-Leaf (6 switches, 8 liens entre switches), $8 - 5 = 3$ liens sont donc bloqués ; pour le Fat-Tree (20 switches, 32 liens entre

switches), $32 - 19 = 13$ liens le sont (calcul théorique, non mesuré). Les 0 % de perte observés prouvent que la topologie fonctionne sans boucle, mais la redondance n'est pas exploitée en parallèle : c'est exactement l'objectif du TP (construire la topologie, pas exploiter tous les chemins).

7 Synthèse

Ce que j'ai construit	Ce que cela montre	Mécanisme utilisé
Single, Linear, Tree (<code>mn --topo</code>)	Lire une topologie avec <code>nodes</code> , <code>net</code> , <code>links</code> , <code>dump</code> ; chemins qui s'allongent avec la profondeur	OVSBridge, <code>--controller none</code>
Hiérarchie Core / Distribution / Access	Rôle de chaque niveau ; un seul chemin entre deux branches	API Python, sans STP
Spine-Leaf	Chemins courts et redondants (2 spines) ; nécessité de STP	STPBridge (<code>stp=True</code>) + attente de convergence
Fat-Tree $k = 4$	Multipath entre pods (4 chemins) ; 20 switches pilotés par script	STPBridge, boucles et listes en Python

TABLE 9 – Bilan des topologies étudiées

Réponse à la question fil rouge. Avec Mininet, on décrit une topologie soit en ligne de commande (`--topo single, linear, tree`), soit en Python avec `addHost`, `addSwitch` et `addLink`, ce qui permet d'écrire des architectures régulières (hiérarchique, Spine-Leaf, Fat-Tree) avec quelques boucles. Sans contrôleur, OVSBridge se comporte comme un switch Ethernet classique : les topologies sans cycle fonctionnent directement, tandis que celles qui contiennent des boucles (Spine-Leaf, Fat-Tree) exigent STP et un temps de convergence avant les tests. On valide ensuite chaque topologie avec `nodes`, `net`, `pingall` et des pings ciblés.

Conclusion. À la fin du TP, je sais reconnaître une topologie, la construire avec Mininet, expliquer sa hiérarchie, identifier les chemins possibles et distinguer une topologie simple d'une architecture de datacenter redondante.

8 Grille de validation rapide

Vérification	Commande / observation	OK
Environnement vérifié	<code>mn --version, ovs-vsctl --version,</code> service actif	☒
Test initial sans contrôleur	<code>--test pingall : 0 % de perte,</code> <code>Stopping 0 controllers</code>	☒
Single, Linear, Tree lus et testés	<code>nodes, net, links, dump, pingall</code>	☒
Architecture hiérarchique (Python)	<code>pingall : 56/56, chemin h1 → h8</code>	☒
Spine-Leaf avec STP	<code>pingall : 56/56 après convergence</code>	☒
Fat-Tree $k = 4$ avec STP	<code>pingall : 240/240, chemin h1 → h16</code>	☒

TABLE 10 – Grille de validation

A Sortie complète de `sh ovs-vsctl show` (Fat-Tree $k = 4$)

Listing 12 – Sortie de `sh ovs-vsctl show` : 20 bridges

```
mininet> sh ovs-vsctl show
6e6a476c-0193-4e2c-85a9-7de8c35dd1b2
  Bridge e32
    fail_mode: standalone
    Port e32-eth1
      Interface e32-eth1
    Port e32
      Interface e32
        type: internal
    Port e32-eth4
      Interface e32-eth4
    Port e32-eth3
      Interface e32-eth3
    Port e32-eth2
      Interface e32-eth2
  Bridge e42
    fail_mode: standalone
    Port e42-eth3
      Interface e42-eth3
    Port e42
      Interface e42
        type: internal
    Port e42-eth2
      Interface e42-eth2
    Port e42-eth1
      Interface e42-eth1
    Port e42-eth4
      Interface e42-eth4
  Bridge c1
    fail_mode: standalone
    Port c1-eth3
      Interface c1-eth3
    Port c1-eth2
      Interface c1-eth2
    Port c1
      Interface c1
        type: internal
    Port c1-eth4
      Interface c1-eth4
    Port c1-eth1
      Interface c1-eth1
  Bridge c4
    fail_mode: standalone
    Port c4
      Interface c4
        type: internal
    Port c4-eth1
      Interface c4-eth1
    Port c4-eth3
      Interface c4-eth3
    Port c4-eth4
      Interface c4-eth4
    Port c4-eth2
      Interface c4-eth2
```

```

Bridge e41
  fail_mode: standalone
  Port e41-eth2
    Interface e41-eth2
  Port e41-eth1
    Interface e41-eth1
  Port e41
    Interface e41
      type: internal
  Port e41-eth4
    Interface e41-eth4
  Port e41-eth3
    Interface e41-eth3
Bridge a12
  fail_mode: standalone
  Port a12-eth2
    Interface a12-eth2
  Port a12-eth1
    Interface a12-eth1
  Port a12-eth4
    Interface a12-eth4
  Port a12
    Interface a12
      type: internal
  Port a12-eth3
    Interface a12-eth3
Bridge a22
  fail_mode: standalone
  Port a22-eth3
    Interface a22-eth3
  Port a22-eth2
    Interface a22-eth2
  Port a22-eth1
    Interface a22-eth1
  Port a22-eth4
    Interface a22-eth4
  Port a22
    Interface a22
      type: internal
Bridge a11
  fail_mode: standalone
  Port a11-eth2
    Interface a11-eth2
  Port a11
    Interface a11
      type: internal
  Port a11-eth4
    Interface a11-eth4
  Port a11-eth3
    Interface a11-eth3
  Port a11-eth1
    Interface a11-eth1
Bridge e11
  fail_mode: standalone
  Port e11
    Interface e11
      type: internal
  Port e11-eth2
    Interface e11-eth2
  Port e11-eth1
    Interface e11-eth1
  Port e11-eth4
    Interface e11-eth4
  Port e11-eth3
    Interface e11-eth3
Bridge e31
  fail_mode: standalone
  Port e31-eth1
    Interface e31-eth1
  Port e31-eth3
    Interface e31-eth3
  Port e31-eth2
    Interface e31-eth2
  Port e31-eth4
    Interface e31-eth4
  Port e31
    Interface e31
      type: internal
Bridge e12
  fail_mode: standalone
  Port e12-eth3
    Interface e12-eth3
  Port e12-eth2
    Interface e12-eth2
  Port e12-eth4
    Interface e12-eth4
  Port e12-eth1
    Interface e12-eth1
  Port e12
    Interface e12
      type: internal
Bridge a32
  fail_mode: standalone
  Port a32
    Interface a32
      type: internal
  Port a32-eth2
    Interface a32-eth2
  Port a32-eth1

```

```

        Interface a32-eth1
    Port a32-eth4
        Interface a32-eth4
    Port a32-eth3
        Interface a32-eth3
Bridge a31
    fail_mode: standalone
    Port a31-eth3
        Interface a31-eth3
    Port a31-eth1
        Interface a31-eth1
    Port a31-eth4
        Interface a31-eth4
    Port a31-eth2
        Interface a31-eth2
    Port a31
        Interface a31
            type: internal
Bridge e22
    fail_mode: standalone
    Port e22-eth4
        Interface e22-eth4
    Port e22-eth1
        Interface e22-eth1
    Port e22
        Interface e22
            type: internal
    Port e22-eth3
        Interface e22-eth3
    Port e22-eth2
        Interface e22-eth2
Bridge a41
    fail_mode: standalone
    Port a41-eth1
        Interface a41-eth1
    Port a41
        Interface a41
            type: internal
    Port a41-eth2
        Interface a41-eth2
    Port a41-eth4
        Interface a41-eth4
    Port a41-eth3
        Interface a41-eth3
Bridge e21
    fail_mode: standalone
    Port e21
        Interface e21
            type: internal
    Port e21-eth4
        Interface e21-eth4
    Port e21-eth3
        Interface e21-eth3
    Port e21-eth1
        Interface e21-eth1
    Port e21-eth2
        Interface e21-eth2
Bridge c3
    fail_mode: standalone
    Port c3-eth1
        Interface c3-eth1
    Port c3-eth3
        Interface c3-eth3
    Port c3-eth4
        Interface c3-eth4
    Port c3-eth2
        Interface c3-eth2
    Port c3
        Interface c3
            type: internal
Bridge a21
    fail_mode: standalone
    Port a21-eth3
        Interface a21-eth3
    Port a21-eth1
        Interface a21-eth1
    Port a21-eth4
        Interface a21-eth4
    Port a21-eth2
        Interface a21-eth2
    Port a21
        Interface a21
            type: internal
Bridge a42
    fail_mode: standalone
    Port a42
        Interface a42
            type: internal
    Port a42-eth1
        Interface a42-eth1
    Port a42-eth3
        Interface a42-eth3
    Port a42-eth2
        Interface a42-eth2
    Port a42-eth4
        Interface a42-eth4
Bridge c2
    fail_mode: standalone
    Port c2-eth3

```

```
    Interface c2-eth3
Port c2-eth2
    Interface c2-eth2
Port c2
    Interface c2
        type: internal
Port c2-eth4
    Interface c2-eth4
Port c2-eth1
    Interface c2-eth1
    ovs_version: "3.3.9"
mininet>
```